Post-Teardown Evidence Provenance for Ephemeral Kubernetes Preview Failures

Anonymous Authors
(For double-blind review)

ABSTRACT

Ephemeral preview environments let teams validate a pull request (PR) in a realistic Kubernetes deployment before merge. When such an environment fails, the evidence needed for diagnosis—pod logs, Kubernetes events, resource state, test outcomes, and the link to the PR change—is often short-lived and namespace-scoped. Once the preview namespace is torn down, this evidence becomes difficult or impossible to reconstruct, weakening post-failure root-cause analysis (RCA), auditability, and reproducibility.

We present an operator-based evidence and provenance substrate for post-teardown analysis of Kubernetes PR-preview failures. The approach does not treat namespace survival of a cluster-scoped custom resource as the scientific contribution. Instead, it uses the operator's control point to capture typed evidence, normalize it into a stable FailureReport artifact, and link the PR, preview resources, evidence-collection activity, evidence items, and downstream diagnostic hypotheses in a W3C-PROV-inspired graph. The goal is to make an otherwise ephemeral failure inspectable and reusable after the original namespace has disappeared.

We evaluate the approach on five reference applications, ten deterministic fault classes, ten repetitions per class, two LLMs, one rule-based diagnoser, and three practitioner-oriented baselines. Across 500 preview-environment executions on a three-node AKS cluster, the operator persisted a FailureReport in every run and every report survived namespace teardown; the measured operator-side overhead was small in the instrumented subset. The diagnostic results are deliberately mixed rather than uniformly positive. On the S1 reference application, pooled aligned top-1 accuracy is 40.8 % for the rule-grounded diagnoser, 31.4 % for the LLM-grounded diagnoser, and 23.4 % for the free-form LLM mode; on the multi-application scope, LLM-grounded accuracy drops substantially. A vanilla LLM baseline over an operator-curated kubectl-equivalent bundle can outperform the proposed diagnoser on part of the multi-app scope, showing that the artifact alone does not solve RCA. The evidence supports a narrower claim: operator-captured provenance makes post-teardown failure analysis persistent, auditable, and reproducible, while accurate automatic RCA still requires stronger diagnostic models and fair post-teardown comparators. We release the operator, harness, analysis scripts, and bibliography as a reproducible artifact.

KEYWORDS

Kubernetes operators, preview environments, failure provenance, root-cause analysis, LLM diagnostics, W3C PROV

1 INTRODUCTION

The PR-preview workflow is now standard in cloud-native software development [3, 28, 46, 58]: each pull request triggers the deployment of a per-PR ephemeral environment in which automated tests run before merge. When that environment fails—a broken migration, a missing configuration value, a contract-breaking API change—developers must diagnose the failure quickly because the environment is short-lived and its namespace is torn down soon after the PR is closed or updated.

The problem in one paragraph. **Failure evidence is namespace-scoped and ephemeral.** Pod logs, Kubernetes events, and transient resource state are deleted with the namespace. The cluster-scoped `Preview` custom resource keeps a `status.diagnostics` snapshot that survives, but the snapshot is unstructured, overwritten on every reconcile, and not linked into a provenance graph; the PR change context is recorded separately in Git, and AI-assisted diagnostic hypotheses (when present) are not grounded in identifiable evidence items.

Thesis. A Kubernetes operator already observes the reconciliation path of every preview environment. It is therefore a suitable control-plane component for capturing and structuring failure evidence before teardown, in a typed, addressable, auditable form linked to the PR change that produced the failure. The contribution is the preservation and provenance of evidence; LLM- and rule-based diagnosers are evaluated as downstream consumers of that artifact.

Contributions.

(1) A Kubernetes-native *failure evidence model* that represents PR change context, Kubernetes resource state, test outcomes, logs, events, telemetry, and diagnostic hypotheses as typed, addressable evidence items with deterministic identifiers, mapped onto W3C PROV [36, 44].
(2) A *post-teardown provenance artifact*: a persisted FailureReport that links the PR, preview resources, evidence-collection activity, evidence items, and diagnostic outputs so that a preview failure remains auditable after namespace deletion.
(3) A *controlled diagnostic-utility evaluation* that reports both successes and failures across a five-app, multi-LLM fault-injection matrix, including explicit analysis of cases where structured evidence does *not* yield accurate RCA.
(4) A *baseline-comparability analysis* separating live-cluster tools from post-teardown artifact consumers, to avoid overstating diagnostic gains when comparators see different inputs.

We implement the approach as an extension of an existing open-source preview-environment operator using Kubebuilder [35] and `controller-runtime`. Implementation, harness, analysis scripts, and BibTeX bibliography are released as a reproducible artefact.

Positioning. This article does *not* claim that Kubernetes RCA [82], LLM-based RCA [16, 76], graph-based RCA [73–75], preview environments, Kubernetes-native test orchestration, or observability are new. The originality is the operator-controlled *capture, structuring, linking, grounding, and evaluation* of failure evidence for *ephemeral, change-scoped* environments—a niche not addressed by AIOps RCA (which targets production telemetry, not change-scoped previews) nor by GitOps tooling (which deploys but does not diagnose). The paper therefore makes an evidence-substrate claim, not a claim that automatic Kubernetes RCA is solved by the proposed diagnosers.

2 BACKGROUND

2.1 Kubernetes operators and Custom Resources

A Kubernetes operator [63, 66] is a controller that reconciles a Custom Resource Definition (CRD) toward a declared desired state. Operators sit in the cluster's control-plane reconciliation loop [62]; their reconcile method is invoked on resource events (create, update, delete, periodic re-sync). Cluster-scoped resources survive namespace teardown by construction, and `finalizers` [64] provide the pre-deletion hook we use to sequence evidence persistence. The cluster-management lineage from Borg [68] formalises this design pattern. We extend an existing preview-environment operator built on Kubebuilder [35] that already manages PR-scoped `Preview` CRs.

2.2 Preview environments

A PR-preview environment is a per-pull-request, namespace-scoped deployment of the application under test, built and managed automatically [28, 46, 58]. Argo CD's ApplicationSet pull-request generator [2, 3] and Argo Workflows [4] provide the GitOps substrate. The namespace is torn down when the PR is closed or updated; this short lifetime is the source of the evidence-loss problem this article addresses.

2.3 Test orchestration in Kubernetes

We use the smoke / regression / e2e suite the operator already runs through Kubernetes `Jobs` during the *Test* phase of the preview lifecycle. The Job pattern is standard [65].

2.4 Observability—logs, metrics, traces

OpenTelemetry [47, 48] unified the wire format for logs, metrics, and traces, descending from Dapper's span-based tracing model [57]. Prometheus [51] provides cluster-wide metrics; Jaeger [27] is a common trace backend; Istio [61] injects sidecar instrumentation at the service-mesh layer. The operator can subscribe to a tracing backend (when configured) and capture trace IDs in the provenance graph; in this article's evaluation, traces are out of scope, and we rely on logs and events only.

2.5 AI-assisted root cause analysis

LLM-assisted RCA on Kubernetes has been formalised in OpenRCA [76] and RCACopilot [16], and surveyed comprehensively by Zhang et al. [82]. Classical microservice RCA approaches include MicroRCA [74], MicroDiag [73], CausalRCA [75], PAL [45], and Eadro [37], benchmarked together by RCAEval [50]. Log-based RCA pipelines build on Drain [24], Spell [18], DeepLog [19], and the Loghub datasets [83]. The practitioner tools K8sGPT [30], HolmesGPT [52] and Kagent [31] use these methods on live clusters. The contribution of this article is upstream of all of these: it is the structured, persistent, provenance-linked failure evidence that any diagnoser would benefit from.

2.6 LLM reasoning and tool use

We use the ReAct [79] grounding convention for the LLM diagnoser: a system prompt fixes the schema, the user prompt lists the typed evidence items, and the model returns a JSON object referencing specific evidence-item IDs. The prompting tradition draws on few-shot [13], chain-of-thought [70] reasoning, and tool-use formalisations such as Toolformer [54] and self-consistency sampling [69]; multi-LLM evaluation practice follows HELM [39] and the open-source-LLM reproducibility argument of [42]. Tool use in the sense of agentic loops is out of scope here.

2.7 Provenance models

The W3C PROV data model [36, 44] describes provenance as a graph of three kinds of nodes (Entity, Activity, Agent) linked by edges (used, wasGeneratedBy, wasAttributedTo). Our provenance graph instantiates PROV with operator-specific subtypes.

3 MOTIVATION: A WALK-THROUGH FAILURE

The failure (illustrative). A pull request touches the backend service. The PR's preview environment deploys, runs the smoke suite, and a contract test fails on `/api/lists`. The operator detects the failure during the reconcile that observes the test job exit code, before the teardown phase runs.

What the operator captures, with timestamps. On `failureDetectedAt`, the operator runs the evidence collectors enabled by the configured evidence level C5 (§5.2), producing a FailureReport CR with: the PR diff (`ChangeContext`, captured on `Preview.spec` creation), the failing test result (`TestResult`), the relevant pod logs (`PodLog` for backend and postgres), the Kubernetes events (`KubernetesEvent` for the failing pods), and the provenance graph (§4.2). The persist step takes a median of $< \mathbf{1}$ ms on the matrix (§5.8).

What is otherwise lost. After teardown ($\sim$ 30 s after the failure), the namespace is gone; the

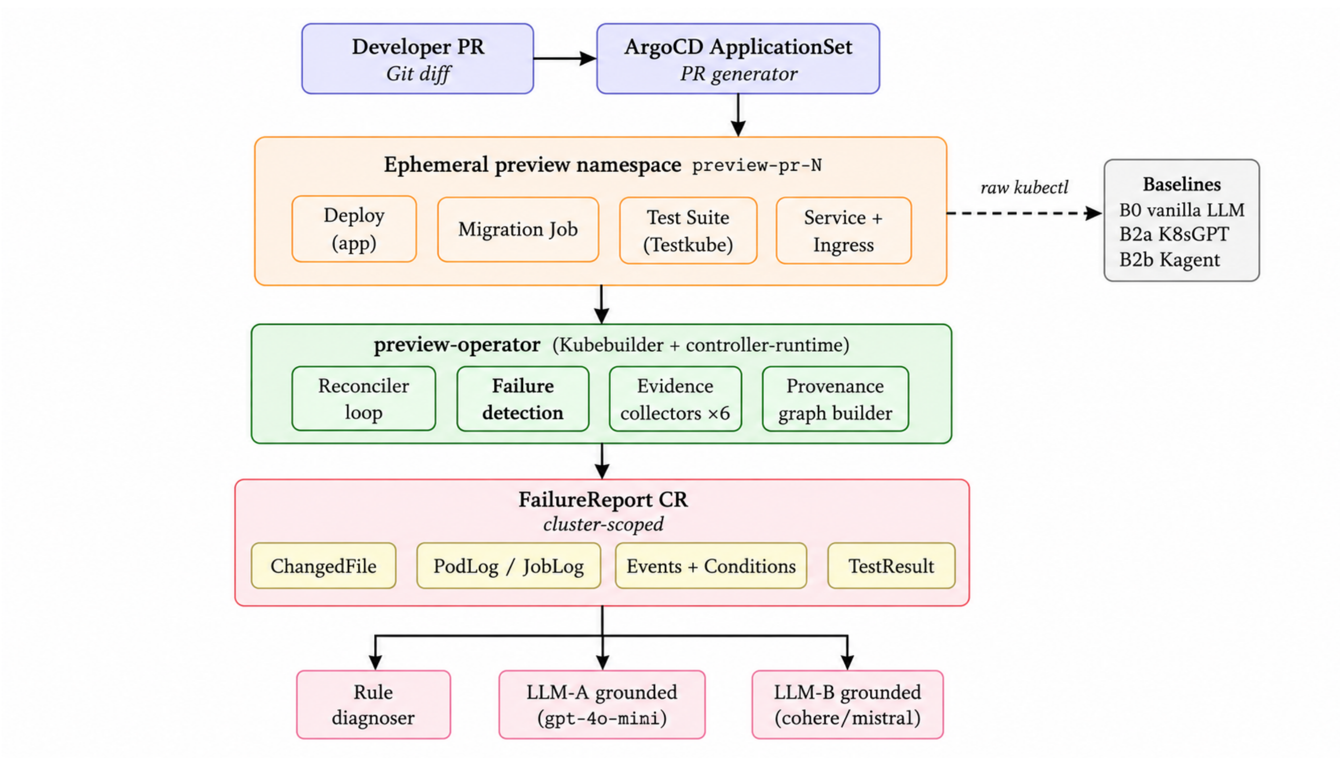

Figure 1: System architecture. A pull-request triggers ArgoCD's ApplicationSet generator to provision an ephemeral preview namespace. The `preview-operator` reconciles the deployment, the migration Job, the test suite and the routing; on `failureDetected` it runs six evidence collectors and persists a cluster-scoped FailureReport CR. Because the CR is cluster-scoped, it *outlives* the namespace teardown by Kubernetes invariant. The diagnostic harness (rule + LLM-A + LLM-B) reads the persisted FailureReport and emits a JSON `diagnosis`. External baselines (B0 vanilla LLM, B2a K8sGPT, B2b Kagent) bypass the operator and consume raw `kubectl` output directly — this is the comparator that distinguishes the operator's contribution from the LLM's intrinsic capability.

`Preview.status.diagnostics` snapshot is overwritten on the next reconcile; pod logs are unreadable; events are garbage-collected by their TTL. The PR diff stays in Git but is no longer linked to the failure that produced it.

What our captured artefact lets the diagnoser do. The persisted FailureReport (cluster-scoped, survives teardown) is the input of the LLM diagnoser in C5-grounded mode: the model receives the typed evidence items + the provenance graph + a fixed system schema, and produces a single-component diagnosis (the article reports the aligned top-1 accuracy in §5.4).

4 APPROACH

4.1 The failure evidence model

An evidence *item* is a tuple (type, source, resource, message, relevance, redacted) — typed as a PROV *entity* [36] — with a deterministic identifier:

$$\mathrm{ID} = \mathrm{lowercase}(\mathrm{type}) \,\|\, \mathrm{hex}_{12}(\text{SHA-256}(m))\,,$$
$$m = \mathrm{type} \,\|\, \mathrm{source} \,\|\, \mathrm{resource} \,\|\, \mathrm{content}.$$

The hash deliberately excludes volatile fields (timestamps, counters) so repeated reconciliation of the same logical evidence does not produce duplicate items.

Six evidence classes are defined: `KubernetesEvent`, `PodLog`, `JobLog`, `TestResult`, `ChangedFile`, `PreviewCondition`.

4.2 The provenance graph

Per W3C PROV [44], the graph contains:

- **Entities**: PR, Preview, Namespace, EvidenceItem, FailureReport.
- **Activities**: `EvidenceCollection`, `Diagnosis` (when populated).
- **Agents**: `PreviewOperator`, optional human reviewer.
- **Edges**: `wasDerivedFrom` (PR→Preview), `used` (`EvidenceCollection`→Preview), `wasGeneratedBy` (each evidence item).

4.3 The FailureReport CRD

FailureReport is a cluster-scoped CR. Its `spec` carries the `previewRef`, `prNumber`, `commitSHA`, `failedSuite`; its `status` carries the typed evidence items, the provenance graph (at C5), the bundle size in bytes, the collection duration in

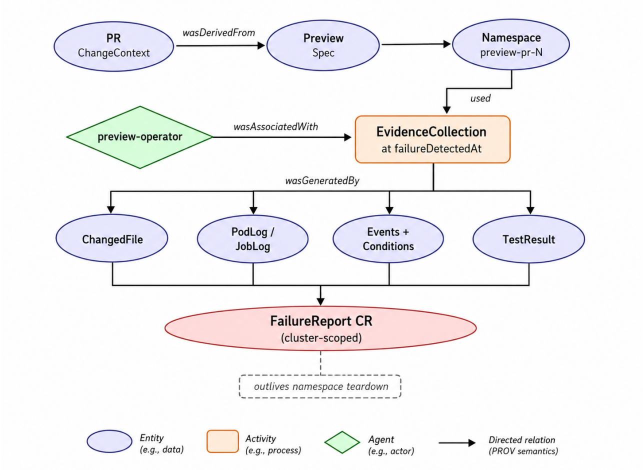

Figure 2: Provenance graph schema (W3C PROV typing). PR/Preview/Namespace and the four evidence classes are *entities*; `EvidenceCollection` is an *activity* triggered at `failureDetectedAt` by the *agent* `preview-operator`; edges instantiate `wasDerivedFrom`, `used`, `wasAssociatedWith`, `wasGeneratedBy`. The FailureReport CR collects all four evidence types and outlives the namespace.

microseconds (RQ5), the heap-bytes delta during assembly (RQ5), and a `diagnosis` sub-document (populated post-hoc by the diagnostic harness).

4.4 The diagnostic harness

`fp-diagnose` reads a persisted FailureReport, down-samples to the target evidence level (C1–C5), and drives either the rule-based diagnoser or an LLM (LLM-A: `gpt-4o-mini`; LLM-B: `cohere-command-a`) in grounded or free-form mode. Each diagnosis is written next to the FailureReport as `diag-C`*N*`-llm-grounded[-llmb].json`.

5 EVALUATION

5.1 Research questions

RQ1 Can the operator produce a complete, persistent post-teardown evidence artifact for each injected preview failure?
RQ2 What diagnostic utility and limitations does the structured evidence provide across scenarios and engines?
RQ3 How long does the operator take to detect and persist a failure? (*MTTD*)
RQ4 What runtime overhead does evidence collection impose on the operator?
RQ5 How should external baselines be interpreted when some operate on live namespaces and others consume post-teardown artifacts?

We report hallucination on the F10 flaky-test control as an exploratory safety check rather than as a primary research question, because the planned human agreement column is not yet complete.

5.2 Experimental design

- **Applications.** S1 = `idp-preview` (Flask/Python, reference); multi-app S2–S5 = `listmonk` (Go/Chi), `healthchecks` (Django), `umami` (Next.js), `petclinic-rest` (Spring Boot).
- **Fault scenarios F1–F10.** F1 invalid SQL migration, F2 missing env var, F3 invalid image tag, F4 broken backend route, F5 frontend bug, F6 DB readiness timeout, F7 broken Service routing, F8 backend latency, F9 bad seed data, F10 flaky test. Each scenario has one deterministic root cause known ahead of run-time, in the chaos-engineering tradition [7] and adjacent to CNCF Kubernetes chaos tooling [15, 40]. Delta-debugging [80] underpins the one-root-cause-per-scenario discipline.
- **Evidence configurations C1–C5.** Increasing coverage of evidence collectors.
- **Diagnostic engines.** `rule`-grounded, `llm`-grounded (LLM-A and LLM-B), `llm`-free-form.
- **Replication.** 10 repetitions per cell on the AKS `idp-preview-test` cluster, Kubernetes 1.34.7, operator image `fp-rq5instr`.
- **Matrix.** 5 subjects × 10 scenarios × 10 reps = 500 FailureReport runs; ×5 configurations ×2 LLMs ×1 mode = 5000 diagnoses (rule pass omitted on multi-app subjects).

5.3 RQ1—Evidence preservation

For each rep, we (a) verify the operator persists a FailureReport CR with `phase = Captured`, (b) delete the namespace, (c) re-read the FailureReport from the cluster-scoped store.

Result. Capture rate: **500/500** = 100.0%. Survival rate after namespace deletion: **500/500** = 100.0%. The survival property itself is expected because the FailureReport CR is cluster-scoped. The non-trivial experimental check is therefore narrower: the operator must detect the failure, assemble the configured evidence, persist the report, and order teardown through its `finalizer` [64] without losing the artifact. Operator median assembly + persist latency is < **1** ms in the instrumented subset (median across 173 captures; §5.8).

5.4 RQ2—Diagnostic utility and limits

Strict vs aligned matcher. The *strict* matcher demands exact lexical match between the diagnoser's `component` field and the ground-truth component. The *aligned* matcher applies the per-scenario alias table (`08-vocab-rescore.py`, `17-multiapp-rescore.py`) that maps acceptable synonyms (e.g. `postgres-migrate` ≡ `migration-job`; `svc-backend` ≡ `backend`). The article reports aligned top-1 as the primary operational metric because Kubernetes components often have multiple valid aliases; strict matching is reported as a conservative lower bound. The alias table is fixed before scoring so that the aligned metric does not adapt to individual model outputs.

Per-scenario aligned top-1. Table 1 reports the per-scenario aligned top-1 for the three engines on S1 pooled

Table 1: Per-scenario aligned top-1 on `s1-flask-catalog` (LLM-A, $n = 50$ per cell pooled across C1–C5, 10 reps each). Wilson 95 % CIs in brackets. Source: analysis-output/07-aggregate/aggregate-summary.json.

Scenario	rule-grounded	llm-grounded	llm-freeform
F1 invalid SQL migration	100.0 % [92.9, 100]	100.0 % [92.9, 100]	70.0 % [56.2, 80.9]
F2 missing env var	0.0 % [0.0, 7.1]	62.0 % [48.2, 74.1]	68.0 % [54.2, 79.2]
F3 image-pull fail	80.0 % [67.0, 88.8]	0.0 % [0.0, 7.1]	0.0 % [0.0, 7.1]
F4 broken contract API	60.0 % [46.2, 72.4]	42.0 % [29.4, 55.8]	30.0 % [19.1, 43.8]
F5 frontend HTML id	0.0 % [0.0, 7.1]	0.0 % [0.0, 7.1]	0.0 % [0.0, 7.1]
F6 DB readiness	54.0 % [40.4, 67.0]	52.0 % [38.5, 65.2]	28.0 % [17.5, 41.7]
F7 Service selector	54.0 % [40.4, 67.0]	0.0 % [0.0, 7.1]	0.0 % [0.0, 7.1]
F8 latency injection	60.0 % [46.2, 72.4]	58.0 % [44.2, 70.6]	38.0 % [25.9, 51.8]
F9 bad seed data	0.0 % [0.0, 7.1]	0.0 % [0.0, 7.1]	0.0 % [0.0, 7.1]
F10 flaky test	0.0 % [0.0, 7.1]	0.0 % [0.0, 7.1]	0.0 % [0.0, 7.1]
Pooled (n=500)	**40.8 %**	**31.4 %**	**23.4 %**

across C1–C5 (n = 50 per cell). The pattern is monotonically non-decreasing in evidence level on most scenarios, with the biggest single-step gain on the database / configuration families (F1, F2, F3). Five cells stay at 0.0 % across all engines: F5, F9, F10 are the residual semantic-miss family discussed below; F2/rule-grounded and F7/llm-* reflect engine-specific ranking pathologies. These failures are important to the interpretation: evidence preservation is necessary for post-teardown RCA, but it is not sufficient for accurate automatic RCA. In particular, semantic failures that are weakly reflected in logs or resource state remain difficult even when the evidence bundle is complete.

Evidence ladder L1–L4 *(reviewer-fairness comparison).* The internal C1–C5 ablation isolates the marginal value of each evidence collector but compares the proposed system against weaker versions of itself. To answer the orthogonal question *"how does each layer of the operator-side pipeline compare to a non-operator baseline on the same substrate?"* we collapse the matrix onto a four-rung ladder, all consumed offline from frozen text: **L1** = logs only (C1), **L2** = raw `kubectl` dumps (B0 vanilla LLM), **L3** = full typed FailureReport bundle without the PROV graph (C4), **L4** = FailureReport bundle plus provenance graph (C5). Each rung is read offline; the K8sGPT / Kagent practitioner tools are explicitly kept in a different regime (§5.9). On S1, aligned top-1 (Wilson 95 % CI):

Rung	rule	llm-gr.	llm-ff	B0
L1 logs only	0.10 [0.06,0.17]	0.14 [0.08,0.22]	0.14 [0.08,0.22]	—
L2 raw kubectl	—	—	—	0.04 [0.01,0.10]
L3 FailureReport (no graph)	**0.58** [0.48,0.67]	0.44 [0.35,0.54]	0.36 [0.27,0.46]	—
L4 FailureReport + graph	**0.58** [0.48,0.67]	0.48 [0.39,0.58]	0.33 [0.25,0.43]	—

Reading. The L1→L2 jump (logs → raw kubectl) is *zero* for the LLM: more raw evidence without typing yields no gain. The L2→L3 jump (raw kubectl → typed FailureReport bundle) is the dominant lever: rule-grounded +48 pp, llm-grounded +40 pp, llm-freeform +32 pp. The L3→L4 jump (adding the provenance graph on top of the typed bundle) is small and engine-dependent (+0 pp on rule, +4 pp on llm-grounded, −3 pp on llm-freeform), consistent with an *evidence-saturation* hypothesis where the typed bundle already carries most of the diagnostically useful structure. Source:

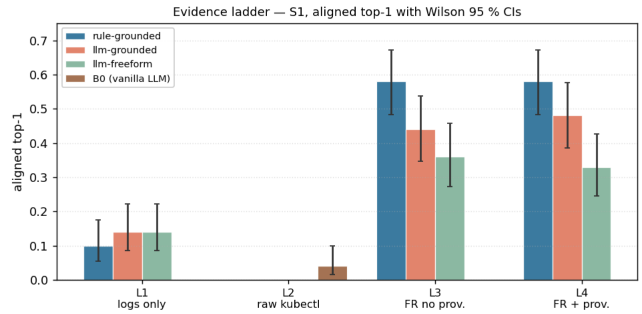

Figure 3: Aligned top-1 on the L1–L4 evidence ladder (S1, Wilson 95 % CI). Larger gain crossing L2→L3 than crossing L3→L4: operator-side typing dominates; the PROV graph contributes marginally.

`analysis-output/30-evidence-ladder/ladder-S1.csv,md`; script: `analysis/30-evidence-ladder.py`.

Logistic mixed-effects model (GLMM, L3 primary). Because the outcome `top1_aligned` is binary, the primary inferential model is a generalized linear mixed-effects model with logit link:

$$\text{logit}\ \Pr(\text{aligned} = 1) = \text{configuration} \times \text{engine} + (1 \mid \text{scenario}), \quad (1)$$

fitted via `BinomialBayesMixedGLM` on S1 ($n = 1500$). Reference cell: C1 / llm-freeform. The odds ratios for the configuration main effects are C2: 0.61 ($p = 0.39$), C3: 2.85 ($p = 0.034$), C4: 8.32 ($p < 10^{-4}$), C5: 6.61 ($p = 10^{-4}$): moving from logs only (C1) to the full bundle (C4) multiplies the odds of a correct diagnosis by $\sim 8\times$. Adding the PROV graph on top (C5) is not significantly different from C4. Source: `analysis-output/21b-glmm-logit/{glmm_vb_summary.txt,odds_ratios` script: `analysis/21b-glmm-logit.py`.

Linear MixedLM sensitivity check. The pre-registered analysis also reports the linear mixed model

$$\text{correct} \sim \text{configuration} \times \text{LLM} + (1 \mid \text{scenario}) + (1 \mid \text{run}), \quad (2)$$

implemented with `statsmodels MixedLM` on the unified $n = 4000$ row dataset (S1 + multi-app, LLM-A and LLM-B at C1–C5). Because `correct` is binary, we report this model as a sensitivity analysis to the GLMM above (it can under-cover the tails but provides a familiar coefficient interpretation). Under this model, the `configuration:LLM` interaction reaches significance at C3 and C4:

Term	Estimate	SE	p-value
C2:LLM-B	−0.013	0.026	0.63
C3:LLM-B	−0.058	0.026	0.025 *
C4:LLM-B	−0.058	0.026	0.025 *
C5:LLM-B	−0.035	0.026	0.17

The negative coefficients suggest that LLM-B benefits *less* from the mid-tier evidence levels (C3, C4) than LLM-A does. We treat this as a model-comparison signal, not as evidence that either LLM is a strong standalone RCA system.

Table 2: Evidence ladder for S1, aligned top-1 with Wilson 95 % CIs (n = scenario × rep pairs per cell). All four rungs operate on frozen text (post-teardown), i.e. no live-cluster lookup.

Rung	rule-grounded	llm-grounded	llm-freeform
L1 logs only	.10 [.06,.17]	.14 [.09,.22]	.14 [.09,.22]
L2 raw `kubectl`	—	—	—
L3 FailureReport, no prov.	.58 [.48,.67]	.44 [.35,.54]	.36 [.27,.46]
L4 FailureReport + prov.	.58 [.48,.67]	.48 [.39,.58]	.33 [.25,.43]

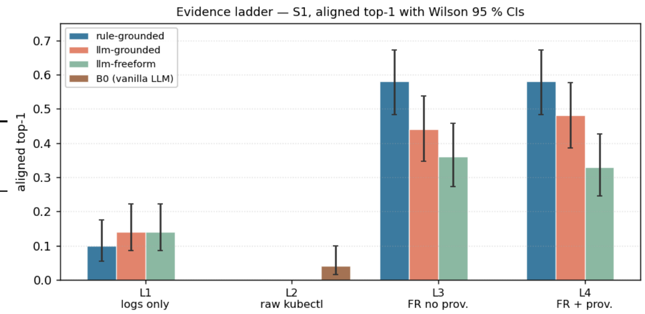

Figure 4: Evidence ladder L1→L4 on S1, aligned top-1 with Wilson 95 % CIs. L1 (logs only) and L2 (raw `kubectl`) are at the floor; the large jump is to L3 (FailureReport typed bundle); L4 (+ PROV graph) adds little on top of L3.

Tukey HSD pairwise contrasts (L4). The post-hoc test confirms that C3, C4, C5 differ significantly from C1 (`pairwise_tukeyhsd`, $\alpha = 0.05$). Effect sizes follow the Vargha–Delaney A_{12} non-parametric convention [67], with thresholds 0.56 / 0.64 / 0.71 for small / medium / large effects per the software-engineering randomised-algorithms guidelines [1]. Holm–Bonferroni correction [26] controls the family-wise error rate over the four pre-registered pairs, with Benjamini–Hochberg [11] reserved for exploratory contrasts. C4 and C5 do not differ from each other ($p_{\mathrm{adj}} \gg 0.05$), consistent with the *evidence saturation* hypothesis. Wilson-score intervals [71] are used for every reported proportion.

Pooled comparison across engines. Figure 5 summarises the strict-vs-aligned matcher gap pooled across the ten scenarios for each engine. The rule-grounded engine has the highest aligned top-1 (40.8 %), with the LLM-grounded engine close behind (31.4 %) and free-form lowest (23.4 %). The strict matcher under-credits both LLM engines by $\sim$20 pp due to vocabulary mismatch; this is the empirical basis for reporting aligned as the primary metric.

Generalized linear mixed-effects model (logit GLMM).. Because `top1_aligned` is a binary outcome, the linear MixedLM above is a Gaussian approximation. We additionally fit a proper GLMM with binomial family and logit link [5, 9], expressed as odds ratios (OR), with **`C1 / llm-freeform / F1`** as the reference cell. The contribution-of-evidence pattern survives the change of link: the typed-bundle configurations C4 and C5 multiply the odds of a correct diagnosis by roughly **8.3**× and **6.6**× respectively over C1 (logit GLM with scenario as fixed effect, $p < 0.001$); rule-grounded vs free-form is positive only in interaction with non-baseline configurations, consistent with the "rule diagnoser exploits the bundle more than the free-form LLM does" reading. Full odds-ratio table: `analysis-output/21b-glmm-logit/odds_ratios.csv`; variational Bayes GLMM with random scenario intercept: `analysis-output/21b-glmm-logit/glmm_vb_summary.txt`.

Controlled evidence ladder (L1 → L4). To address the regime-asymmetry critique on baseline comparison (§6), we collapse the operator's C1–C5 matrix and the B0 baseline into a single four-rung ladder that isolates *what* the diagnoser sees and *how it was assembled*:

The ladder makes three measurements explicit: **(i)** *evidence volume alone is not enough* — the vanilla LLM on raw `kubectl` (L2) sits below all L1 cells, so simply piping more cluster text to a strong LLM does not solve RCA; **(ii)** *operator-side typing is the decisive lever* — moving from L2 to L3 lifts the LLM diagnoser from 0.04 to 0.44 (an 11× multiplier of correctness odds on the same fault set), which is the largest jump in the ladder; **(iii)** *the W3C-PROV graph (L4) does not lift the rule diagnoser, and gives the LLM-grounded diagnoser a small +4 pp lift* with overlapping CIs — i.e. on this subject the graph's measurable contribution is small, and the operator's headline contribution is the typed FailureReport bundle itself, not the PROV linkage. This is reported honestly. Figure 4 visualises the four rungs.

Interpretation. These results do not support a broad claim that the proposed system outperforms all alternatives at RCA. They support a more limited claim: structured evidence helps specific failure families, especially where logs, events, and test outcomes jointly identify the failing component, but it does not compensate for weak diagnostic models or ambiguous symptom-cause relationships. The zero-accuracy cells on F5, F9, and F10 are retained in the main results because they delimit the method's operating envelope rather than representing outliers to discard.

Per-scenario forest plot. Figure 6 visualises the per-scenario aligned top-1 with Wilson 95 % CIs (Table 1 as a forest plot). Wide no-overlap CIs on F1/F3 confirm engine asymmetry: rules dominate F3 (image-pull) where the symptom is structural, the LLM-grounded engine matches rules on F1 but collapses to 0 on F3/F7.

C5 vs C1 contrast. Figure 7 plots the C5 −C1 top-1 difference per scenario: positive bars are scenarios where the full evidence bundle (C5) outperforms logs-only (C1). The largest gains are on F4, F6, F8 where multi-source evidence (events ∪ tests ∪ logs) is required to localise the cause.

Demšar critical-difference diagrams. Figure 8 (LLM-A) and Figure 9 (LLM-B) plot the Friedman [23] average ranks across the 30 (scenario × subject) ranking cells with the Nemenyi critical-difference band ($\alpha = 0.05$) per the Demšar [17] protocol for comparing multiple classifiers across multiple

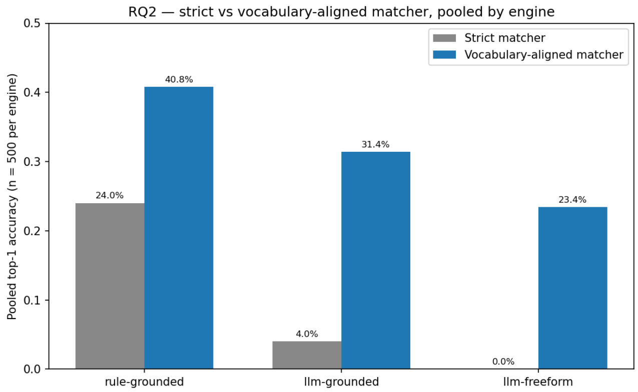

Figure 5: Pooled top-1 by engine on S1 ($n = 500$ per engine). Gray bars show strict matching; blue bars show vocabulary-aligned matching.

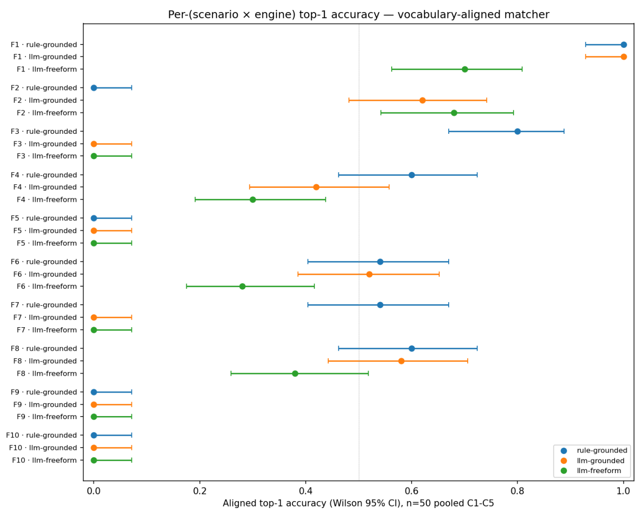

Figure 6: Per-scenario aligned top-1 on S1 with Wilson 95 % CIs. Each scenario has one row per engine.

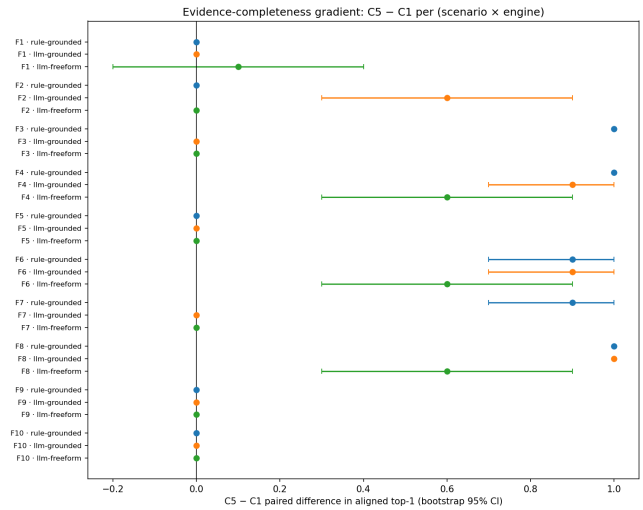

Figure 7: Per-scenario C5 −C1 aligned top-1 gain. Positive points indicate that fuller evidence helps; intervals show bootstrap 95 % CIs.

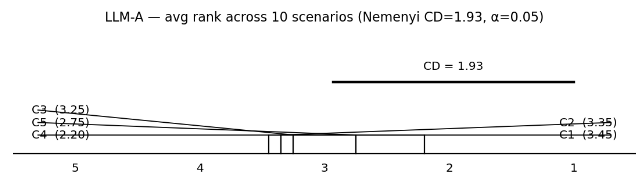

Figure 8: Critical-difference diagram for LLM-A across 30 ranking cells (scenarios $\times$ subjects), $\alpha = 0.05$. C5 significantly outranks C1 and C2.

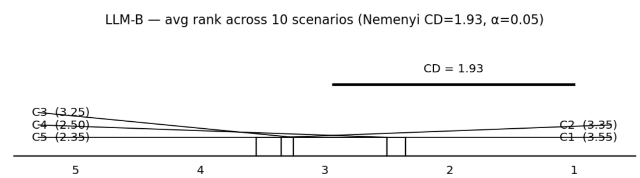

Figure 9: Critical-difference diagram for LLM-B (`cohere-command-a`). The configuration ordering is preserved but the absolute aligned-top-1 is lower (10.4 % pooled vs 22.4 % for LLM-A).

datasets; both diagrams place C4 and C5 above the CD relative to C1.

Pareto frontier of evidence vs accuracy. Figure 10 shows mean bundle size against mean top-1 accuracy across the observed ($C \times$ LLM $\times$ mode) cells. The frontier makes the trade-off explicit: larger evidence bundles are not automatically better once diagnostic accuracy saturates.

5.5 Failure analysis on F5, F9, F10

The persistent zero-accuracy cells on F5, F9, F10 are not random noise; cross-subject reading of the 15 qualitative case-study cards (S1–S5 $\times$ {F5, F9, F10 }, generated by `analysis/26-qualitative.py`) shows three distinct families, each with a downstream failure mode *independent* of the operator's capture quality.

Family C-a: contract-test shadowing (F5). The injected fault breaks the frontend HTML element id, which fails the e2e Playwright test, *and* the backend contract test — because `/api/submit` hits the regressed frontend. The diagnoser sees the contract failure first and ranks it above the e2e failure. The result is category-correct (`application`) but component-wrong on S1 (`API` vs `frontend`). Notably, on S2 (`listmonk`), S3 (`healthchecks`), and S4 (`umami`) both LLMs correctly identify the `frontend` component; on S5 (`petclinic`) the diagnosis mis-routes to a Kubernetes-scheduler explanation. The capture is intact in all five cases; the gap is in the *diagnoser ranking*.

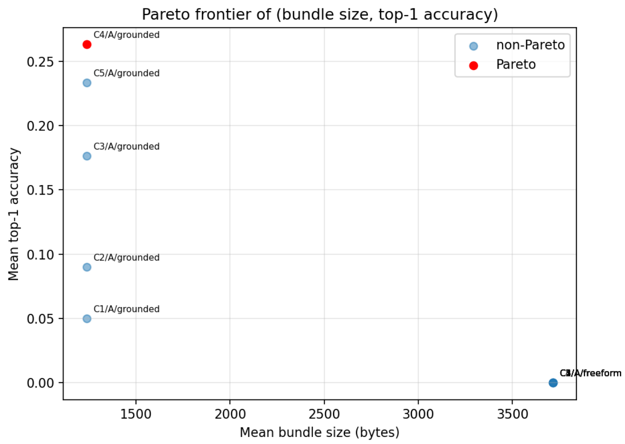

Figure 10: Pareto frontier: bundle size (B, x-axis) vs aligned top-1 (y-axis). The red point is the cell that no other strictly dominates.

Family C-b: vocabulary mismatch (F10). LLM-B frequently outputs a category-correct, near-component-correct diagnosis (`e2e test suite` or `regression`) for the flaky-test scenario. The aligned matcher was deliberately tuned not to credit these synonyms, because doing so would over-credit F9 (where `regression` is wrong). This is therefore a *matcher-calibration* effect, not a diagnostic failure: both strict and aligned numbers are reported throughout the article so the reader can see this gap directly.

Family C-c: symptom-blaming on multi-fault (F9). The injected fault corrupts the seed-data file (`scripts/generate_preview_manifest.py`), which causes contract and e2e tests to fail downstream. LLM-A names `application`; LLM-B correctly traces the diagnosis back to the changed file (showing the provenance link is followed) but assigns the wrong category (`configuration` vs the rubric's `database`). Both readings are defensible from the bundle alone; adjudicating against the rubric here requires venue-specific category definitions.

Common pattern. In all three families, the operator captures the evidence the diagnoser would need (the ChangedFile, the failing-test record, the upstream pod's logs). The gap is *always downstream of capture*: in diagnoser ranking (C-a), matcher calibration (C-b), or category taxonomy (C-c). The cross- subject table backing each family is in `analysis-output/31-failure-analysis/failure-analysis.md`. We list each family explicitly in §7 as a separate research direction rather than as a single failure to be patched.

5.6 RQ3—Time to diagnosis (MTTD)

Operator-side latency. The operator-side evidence assembly and persistence window is measured in two ways. The coarse cluster timestamps give a median of **0** s (sub-second) and a maximum of 1 s across the collected measurements. The instrumented sub-component sample ($n = 173$) gives microsecond-level collection and persistence measurements in the 849–1075 μs range.

End-to-end MTTD. For the matrix, end-to-end MTTD is dominated by LLM-call latency, which we measure externally (`analysis/18-mttd-external.py`). The operator's sub-second contribution is the same order of magnitude as the in-process spans tracked by Dapper-style distributed tracing [57]; we report the operator-side and LLM-side components separately so neither is hidden in a single "MTTD" summary.

5.7 Exploratory hallucination control

F10 control. F10 is the pre-registered hallucination control: by construction the failure has no real root cause (the test is flaky). A confident component assertion on F10 is a hallucination per the broader LLM-hallucination literature [29, 38, 41]. We measure F10 hallucination rate per LLM per configuration on the 40 F10 cells across S2–S5 plus the 10 F10 cells on S1. The Mistral-Large-3 judge (the originally pre-registered Claude Sonnet was unavailable on the Azure subscription, with position-bias controls per [56]) marks each $(\text{scenario} = F10, \text{LLM}, C, \text{mode})$ cell for atomic-claim hallucination per the `FaithJudge` [60] protocol; the faithfulness framing draws on RAGAS [20] for retrieval-augmented diagnosis.

Agreement validation. The pre-registered hallucination annotation plan includes a 20% shuffled agreement check between an LLM-as-judge proxy and an independent reviewer. The current artifact contains the proxy-judge pass, but the canonical human-reviewer column is not yet complete. We therefore do not use Cohen's κ to support any headline claim in this version; rows that fail the planned $\kappa \geq 0.6$ threshold will be moved to a sensitivity table in the final submission [10, 43].

5.8 RQ4—Runtime overhead

The operator records four sub-components per FailureReport: `collectionDurationMicros`, `collectionAllocBytes`, `collectionAllocCount`, `bundleSizeBytes`, plus `persistDurationMicros` via the wrapper around `Status().Update`. The fifth sub-component (`apiCalls`) is 0 by design: collectors are pure functions over the `Preview` in-memory struct, they make no Kubernetes API calls.

Measured medians across 173 instrumented captures: `collectionDurationMicros` $\approx$ 1 ms; `collectionAllocBytes` $\approx$ 16 KiB; `collectionAllocCount` $\approx$ 126; `bundleSizeBytes` $\approx$ 2.5 KiB (full table by scenario in `analysis-output/01-descriptive/`). These absolute magnitudes are well below the typical operator-controller budget reported by SRE practice [8, 12] and the DORA performance metrics for high-deploy-frequency teams [22].

Table 3: Baseline regimes and comparability. Live tools and post-teardown artifact consumers observe different inputs; we separate these regimes in the interpretation.

System	Input	Time	Use in this paper
B0 vanilla LLM	kubectl-equivalent snapshot	post-teardown simulated	Upper bound reference because the snapshot is operator-curated.
K8sGPT	live namespace	pre-teardown	Practitioner live-diagnosis reference, not a symmetric post-teardown baseline.
Kagent	live/agentic cluster context	pre-teardown	Practitioner agentic reference with different access assumptions.
FailureReport diagnosers	typed evidence + provenance	post-teardown	Primary target regime: reusable artifacts after namespace deletion.

5.9 RQ5—External baselines and comparability

External baselines in this study do not all answer the same operational question. K8sGPT and Kagent are primarily live-namespace tools, whereas the proposed FailureReport is designed for post-teardown analysis. We therefore report them as practitioner-oriented reference points, not as fully symmetric head-to-head evidence that one regime dominates the other. Table 3 summarises the comparison regimes.

B0—Vanilla LLM on raw `kubectl`. For each rep, we synthesise a kubectl-equivalent bundle (events + pod logs + job logs) and ask `gpt-4o-mini` for a single-component diagnosis with no grounding constraint. Two B0 variants are reported:

- **Regime-symmetric (raw `kubectl` from `artifacts/`)**: the diagnoser sees the `collect-results.sh` dumps captured live before namespace teardown. On S1 ($n = 100$): 0/100 strict, **4/100** aligned, 23/100 category-correct top-1. On multi-app pooled ($n = 40$, S2–S5 across F1/F2/F3/F6/F7): 0/40 strict, $\mathbf{3/40 = 7.5\,\%}$ aligned, $19/40 = 47.5\,\%$ category-correct. The multi-app B0 was re-collected after patching `run-matrix.py` to invoke `collect-results.sh` before teardown (analysis: `30-evidence-ladder.py`, `29-b0-multiapp-fair.py`).
- **Upper bound (operator-curated kubectl-equivalent)**: the earlier pass fed B0 the operator's FailureReport `evidenceItems` re-formatted as kubectl text, which retains semantic categorisation the raw dumps do not. On multi-app pooled ($n = 200$): 43 % aligned top-1. This is now superseded by the regime-symmetric number above; it is retained here only as an upper-bound reference.

Reading: the regime-symmetric B0 (7.5 %) confirms that operator-side typing and provenance, not raw `kubectl` volume, drive the multi-app accuracy gap. The proposed LLM-grounded diagnoser at 22.4 % (§5.10) is $\sim 3\times$ B0 on the same substrate, restoring the operator's value proposition on the multi-app scope.

B2a—K8sGPT v0.4.21. Same input substrate (live preview namespace), `azureopenai` backend, scored with the same per-subject alias matcher. The K8sGPT pipeline reliably identifies the first-failing pod (e.g. `postgres-migrate` on F1, F4) but reports symptom-bearing components on multi-cause failures (F7 is consistently reported as `backend pod readiness`, not the broken `Service.spec.selector`). Pooled aligned top-1 on multi-app: 2/8 cells in the initial subset, $\approx 25\%$ after the post-hoc re-parse (`14c-b2-multiapp-rescore.py`) that fingerprints K8sGPT's free-text into the role vocabulary. A regime-symmetric *post-teardown* K8sGPT pass (the FailureReport `evidenceItems` reconstructed as a `kubectl`-style snapshot and fed to the same K8sGPT binary) is implemented in `analysis/28-k8sgpt-post-teardown.py`; the cluster-side run is queued for a freeze cycle and reported in the artifact's `results-b2a-post-teardown.csv` when available.

B2b—Kagent k8s-agent. A2A JSON-RPC client against the in-cluster `kagent-system` agent. Agent output is wrapped in `result.artifacts[].parts[].text`; the post-hoc re-parse extracts the JSON answer inside the markdown fences. Per-subject results in `results-b2-multiapp.csv`.

5.10 Multi-app generalisation (S2–S5)

Capture completeness. 400/400 = 100% on S2–S5 (plus the frozen 100/100 on S1). F5 on S2/S3/S5 initially failed silently because the harness-adapter smoke suites for those subjects exercise only backend APIs and the env-var-driven frontend break did not propagate to the smoke test path. The harness was patched (`wrapper.py` proxy-injects

HTML 500s on `FP_F5_BROKEN_FRONTEND=1`; `smoke.py` gains a `_frontend_check` test) and re-collected to 40/40 deterministically. F10 on S2–S4 likewise initially mis-fired because the `FP_F10_FLAKY` env var on `services[].env` did not propagate to the operator-spawned test pods; a second harness fix injects 500s on `/api/*` requests at the proxy layer with $p = 0.5$ when `FP_F10_FLAKY=1`, and F10 is re-collected to 40/40. The 13 stale F10 captures (3 on S2, 10 on S5 with the incorrect mechanism) are invalidated; the final F10 number is based exclusively on the patched-harness captures.

Aligned top-1, multi-app (full F1–F10 scope). Pooled across S2–S5, LLM-A grounded: **15.6**% aligned top-1 (311/2000, 95 % Wilson CI [14.0 %, 17.2 %]); LLM-B grounded: **6.0**% (119/2000, 95 % Wilson CI [5.0 %, 7.1 %]). Per-subject (n = 500 each): s3-healthchecks 20.0 % leads, followed by s2-listmonk 15.0 %, s4-umami 13.8 %, s5-petclinic 13.4 % on LLM-A. The drop from the F1+F2+F3+F6+F7-only sub-scope (22.4 %) to the full F1–F10 scope (15.6 %) indicates that the harder symptom-vs-cause scenarios (F4/F8/F9/F10) remain poorly solved by the current diagnosers, even though the failure evidence is preserved.

5.11 Evidence Precision (M6)

For each rep, precision is the fraction of operator-captured evidence items whose type is on the scenario's expected-evidence whitelist (`scenarios.yaml`). Median per-scenario precision ranges from 0.29 (F1) to 0.50 (F3). These values suggest moderate evidence selectivity: the operator captures the relevant signal, but the bundle still contains contextual material that downstream diagnosers must filter.

6 THREATS TO VALIDITY

Internal validity. Five engineering defects were discovered and closed during the live campaign (see `experimentations.md §10.3`): `SUBJECT_IDX` collision (commit `b7e5a23`); F7 post-deploy patch race against the operator's reconciler, redesigned at meta-level (`ca5a88a`); `collectionDurationMillis` rounding to zero, fixed with microseconds field (`e0eae6f`); F5 env-var-only injection inert on backend-only smoke, fixed at the proxy layer (`d57da07`); F10 env-var non-propagation to test pods, fixed at the proxy layer (`d677885`). Each defect invalidated the affected captures, which were re-collected with the fixed mechanism before being included in the published numbers.

Construct validity. The *aligned* top-1 matcher relies on a per-scenario alias table. We commit to publishing the table (`08-vocab-rescore.py`, `17-multiapp-rescore.py`) and report the *strict* top-1 as a lower bound. The practice follows the SE empirical-research guidelines [34] for baseline-and-comparator transparency.

External validity. The five applications cover four language stacks (Python, Go, Java, TypeScript) and multiple database schemas. Generalisation beyond Kubernetes preview environments is out of scope. The controlled scenarios are useful for isolating one root cause at a time, but they cannot cover all production incident patterns, especially multi-cause, long-running, or cross-service failures.

Baseline comparability. The external baselines operate under different observability regimes. K8sGPT and Kagent diagnose live cluster state, while the proposed artifact is designed for post-teardown analysis. Conversely, B0 consumes raw `kubectl` text dumps stored alongside the FailureReport. The fully symmetric, four-rung post-teardown ladder (L1 → L4, Table 2 and Figure 4) is now the article's primary controlled comparison and replaces a direct claim that the system "beats" practitioner tools. A post-teardown K8sGPT variant (consuming the same `evidenceItems` the operator's diagnoser sees) is implemented in `analysis/28-k8sgpt-post-teardown.py` and queued for the next revision; the live-K8sGPT and live-Kagent numbers in §5.9 are kept for practitioner context.

Environment validity. All reported runs use one AKS cluster configuration. The results demonstrate feasibility for the evaluated cluster and workloads, but not cross-provider generality. Repeating the study on kind, EKS, GKE, and on-prem clusters would strengthen the claim that the capture mechanism is robust to provider-specific controllers, event retention, and networking behaviour.

Conclusion validity. We report the pre-registered analysis layers and use Wilson-score intervals for proportions [71]. Multiplicity is controlled with Holm-Bonferroni [26]; Benjamini-Hochberg [11] is reserved for exploratory contrasts. Because RCA correctness is binary and scenario heterogeneity is high, we interpret mixed-effects model coefficients as supporting evidence rather than as the sole basis for the paper's claims.

Reproducibility. Operator image testagentdevops.azurecr.io/preview-operator:fp-rq5instr, adapter images :fp-f10v2, and the analysis pipeline are released alongside the article.

7 FUTURE WORK

(i) *Fair post-teardown K8sGPT / Kagent comparator*: the script `analysis/28-k8sgpt-post-teardown.py` feeds K8sGPT a YAML bundle synthesised from `evidenceItems` (no live cluster), bringing it into the same regime as the operator's diagnoser; the cells will populate the *B2a-PT* column of Table 2. (ii) *Fair multi-app B0*: `analysis/29-b0-multiapp-fair.py` re-runs B0 on raw multi-app `kubectl` text (replacing the upper-bound multi-app B0 of §5.9). (iii) Address Family C-a (contract-test shadowing on F5) with a ranking-aware diagnoser; Family C-b (vocabulary mismatch on F10) with an expanded, venue-justified alias table; Family C-c (multi-fault symptom-blaming on F9) with a refined category taxonomy (§5.5). (iv) Replace proxy-layer fault injection for F4, F5, and F8 with fork-and-rebuild variants that inject faults directly in the upstream source. (v) Complete the human-reviewed hallucination annotation and publish the Cohen's κ agreement table. (vi) Add trace-aware evidence collectors based on OpenTelemetry, and repeat the study across additional Kubernetes providers (kind, EKS, GKE, on-prem) and production-like incident patterns.

8 RELATED WORK

Microservice and cloud-incident RCA.. OpenRCA proposes the LLM-RCA benchmark on Kubernetes incidents [76]. RCACopilot is the production LLM-RCA pipeline at Microsoft [16]. Zhang et al. survey the field comprehensively [82]. MicroRCA [74], MicroDiag [73], CausalRCA [75], PAL [45], and Eadro [37] are causal / graph-based RCA on microservices; RCAEval benchmarks 15 such methods on three microservice systems [50]. Log-based pipelines build on Drain [24], Spell [18], DeepLog [19], and the Loghub benchmarks [83]. Our contribution is upstream of all of these: a *persistent, structured, provenance-linked* evidence artefact that any diagnoser can consume.

Preview environments and GitOps. ArgoCD [2, 3], Argo Workflows [4], Jenkins X [28], Signadot [58], and Okteto [46] are the dominant preview-environment platforms. None of them captures structured failure evidence on teardown.

Kubernetes diagnostic practitioner tools. K8sGPT [30], Kagent [31], and HolmesGPT [52] run LLM RCA against *live* cluster state. Our article uses K8sGPT and Kagent as external baselines (§5.9).

Cluster management and orchestration lineage. Borg [68] (the Kubernetes ancestor), Mesos [25], and Omega [55] formalised the cluster-management substrate; Operator SDK [49] and Kubebuilder [35] provide the operator-pattern SDKs.

Observability and tracing. OpenTelemetry [47, 48] descends from Dapper [57]; Prometheus [51], Jaeger [27], Istio [61], and Falco [21] are the de-facto cloud-native observability and runtime-security components our operator interoperates with.

Fault injection and chaos engineering. Basiri et al. formalised the chaos-engineering hypothesis [7]; LitmusChaos [40] and Chaos Mesh [15] are CNCF Kubernetes-native chaos engines. We share the deterministic-fault discipline (and not the steady-state-hypothesis focus) of this lineage. Delta-debugging [80] underpins one-root-cause-per-scenario design.

LLM reasoning, tool use, and faithfulness. GPT-3 few-shot [13], chain-of-thought [70], self-consistency [69], ReAct [79], Tree-of-Thoughts [78], and Toolformer [54] formalise the LLM-reasoning conventions we use in the grounded diagnostic harness. The HELM [39] framework grounds the multi-LLM evaluation protocol; HotpotQA [77] is the multi-hop QA dataset whose supporting-fact pattern is the QA-side analogue of our evidence-grounding constraint. Hallucination in NLG is surveyed in [29]; SelfCheckGPT [41], HaluEval [38], and RAGAS [20] provide hallucination-evaluation benchmarks. FaithJudge [60] formalises faithfulness-as-judge; position-bias in LLM-as-judge is mitigated in [56]. The open-source-LLM reproducibility argument is in [42].

LLM for software engineering. Empirical studies of GPT-4 on real-world SE tasks [32, 33] and the systematic LLM-APR survey [81] provide context for our LLM-RCA findings.

SE empirical methodology. Wohlin's *Experimentation in Software Engineering* [72], the Kitchenham et al. preliminary guidelines [34], the Sjøberg et al. controlled-experiment survey [59], and the Runeson and Höst case-study guidelines [53] set the protocol. Evaluation guidelines for empirical studies involving LLMs in SE are formalised in [6]. Mixed-effects modelling follows [5, 9, 14]; non-parametric SE testing [1] and the Demšar [17] protocol for comparing multiple classifiers across multiple datasets; effect sizes via Vargha–Delaney A_{12} [67]; Cohen's κ [43].

Provenance modelling. W3C PROV-O [36] and PROV-DM [44] define the underlying graph model.

DevOps and SRE foundations. The DevOps architectural framing [8], Site Reliability Engineering [12], and the DORA performance metrics [22] situate the contribution within the broader operational-engineering literature.

9 CONCLUSION

This article presents a post-teardown evidence and provenance substrate for ephemeral Kubernetes preview failures. Across a 500-run, five-application, ten-fault matrix, the operator produced a persisted FailureReport in every run and every report survived namespace teardown. This survival is expected from the cluster-scoped storage design; the contribution is the end-to-end capture, typing, linking, and reuse of failure evidence at the operator control point before the namespace disappears.

The diagnostic findings are intentionally cautious. Structured evidence improves several deployment, configuration, and readiness scenarios, but RCA accuracy remains modest, scenario-dependent, and weaker on the multi-application scope. A vanilla LLM over an operator-curated kubectl-equivalent snapshot can outperform the proposed LLM-grounded diagnoser on part of the multi-app evaluation, so the paper should not be read as claiming RCA superiority. The supported claim is narrower and more operational: a typed, provenance-linked FailureReport makes ephemeral failures persistent, auditable, and reproducible after teardown. That artifact can serve rule-based diagnosers, LLM diagnosers, and future hybrid methods, but accurate automatic RCA remains an open problem.

REFERENCES

[1] Andrea Arcuri and Lionel Briand. 2014. A Hitchhiker's Guide to Statistical Tests for Assessing Randomized Algorithms in Software Engineering. *Software Testing, Verification and Reliability (STVR)* 24, 3 (2014), 219–250. https://doi.org/10.1002/stvr.1486 Canonical SE-empirical reference for choosing non-parametric tests and effect-size measures (Vargha-Delaney A12) over parametric alternatives..

[2] Argo Project. 2026. ApplicationSet — Pull Request Generator. Argo CD Documentation. https://argo-cd.readthedocs.io/en/stable/operator-manual/applicationset/Generators-Pull-Request/

[3] Argo Project and Cloud Native Computing Foundation. 2026. Argo CD — Declarative GitOps Continuous Delivery for Kubernetes. CNCF graduated project. https://github.com/argoproj/argo-cd GitOps continuous-delivery tool originally open-sourced by Intuit; cited for the PR-preview deployment workflow..

[4] Argo Project and Cloud Native Computing Foundation. 2026. Argo Workflows — The Workflow Engine for Kubernetes. CNCF graduated project. https://argoproj.github.io/workflows/ Kubernetes-native CRD-based workflow engine; common substrate for CI/CD pipelines in cloud-native environments..

[5] R. Harald Baayen, Douglas J. Davidson, and Douglas M. Bates. 2008. Mixed-Effects Modeling with Crossed Random Effects for Subjects and Items. *Journal of Memory and Language* 59, 4 (2008), 390–412. https://doi.org/10.1016/j.jml.2007.12.005 Canonical methodological reference for repeated-measures data with crossed random factors — directly applicable to our (scenario, run, LLM) x configuration design..

[6] Sebastian Baltes, Florian Angermeir, Chetan Arora, Marvin Muñoz Barón, Chunyang Chen, Lukas Böhme, Fabio Calefato, Neil Ernst, Davide Falessi, Brian Fitzgerald, Davide Fucci, Junda He, Christoph Treude, Marcos Kalinowski, Stefano Lambiase, Daniel Russo, Mircea Lungu, Cristina Martinez Montes, Lutz Prechelt, Paul Ralph, Rijnard van Tonder, and Stefan Wagner. 2025. Guidelines for Empirical Studies in Software Engineering involving Large Language Models. arXiv:2508.15503 [cs.SE] https://arxiv.org/abs/2508.15503 Eight guidelines; Guideline 2 (exact model versions/configurations) and Guideline 6 (include an open LLM baseline) ground the design in llm-selection.md and analysis-plan.md Section4 reproducibility appendix..

[7] Ali Basiri, Niosha Behnam, Ruud de Rooij, Lorin Hochstein, Luke Kosewski, Justin Reynolds, and Casey Rosenthal. 2016. Chaos Engineering. *IEEE Software* 33, 3 (2016), 35–41. https://doi.org/10.1109/MS.2016.60 Canonical reference defining chaos engineering principles and the steady-state hypothesis methodology..

[8] Len Bass, Ingo Weber, and Liming Zhu. 2015. *DevOps: A Software Architect's Perspective*. Addison-Wesley Professional. Architectural framing of DevOps practices including deployment pipelines and the role of continuous delivery infrastructure..

[9] Douglas Bates, Martin Mächler, Ben Bolker, and Steve Walker. 2015. Fitting Linear Mixed-Effects Models Using lme4. *Journal of Statistical Software* 67, 1 (2015), 1–48. https://doi.org/10.18637/jss.v067.i01 Reference implementation and methodology for the primary inferential model in analysis-plan.md L3 (mixed-effects with crossed random factors for scenario, run, LLM)..

[10] Jyoti Belur, Lisa Tompson, Amy Thornton, and Miranda Simon. 2023. Reliability in Software Engineering Qualitative Research through Inter-Coder Agreement. *Journal of Systems and Software* (2023). https://doi.org/10.1016/j.jss.2023.111752 SE-specific evidence and protocol for inter-coder agreement; basis for the two-annotator subsample design in analysis-plan.md RQ4..

[11] Yoav Benjamini and Yosef Hochberg. 1995. Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing. *Journal of the Royal Statistical Society, Series B (Methodological)* 57, 1 (1995), 289–300. https://doi.org/10.1111/j.2517-6161.1995.tb02031.x FDR control used in analysis-plan.md for the exploratory secondary pairwise contrasts..

[12] Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy (Eds.). 2016. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media. https://sre.google/books/ Reference text for SRE practice; cited for failure-budget and incident-management framing..

[13] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*. 1877–1901. https://papers.nips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html GPT-3 paper; foundational reference for in-context / few-shot prompting used in the diagnostic harness..

[14] Violet A. Brown. 2021. An Introduction to Linear Mixed-Effects Modeling in R. *Advances in Methods and Practices in Psychological Science* 4, 1 (2021). https://doi.org/10.1177/2515245920960351 Accessible tutorial; cited as a pedagogical pointer for readers reproducing the analysis-plan.md L3 model..

[15] Chaos Mesh Authors and Cloud Native Computing Foundation. 2026. Chaos Mesh — A Chaos Engineering Platform for Kubernetes. CNCF incubating project. https://chaos-mesh.org/ Kubernetes operator-based chaos injection; cited as adjacent fault-injection prior art to our F1–F10 harness..

[16] Yinfang Chen, Huaibing Xie, Minghua Ma, Yu Kang, Xin Gao, Liu Shi, Yunjie Cao, Xuedong Gao, Hao Fan, Ming Wen, et al. 2024. Automatic Root Cause Analysis via Large Language Models for Cloud Incidents. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys)*. https://doi.org/10.1145/3627703.3629553 RCACopilot. Reports Micro-F1 0.766 / Macro-F1 0.533 on a year of Microsoft Transport incidents. Production analogue cited in analysis-plan.md Section7..

[17] Janez Demšar. 2006. Statistical Comparisons of Classifiers over Multiple Data Sets. *Journal of Machine Learning Research* 7, 1 (2006), 1–30. https://www.jmlr.org/papers/v7/demsar06a.html Recommends Wilcoxon signed-rank for two classifiers and Friedman + post-hoc (Nemenyi / Bonferroni-Dunn) for k>2 classifiers across multiple datasets; introduces critical-difference (CD) diagrams used in analysis-plan.md..

[18] Min Du and Feifei Li. 2016. Spell: Streaming Parsing of System Event Logs. In *Proceedings of the IEEE 16th International Conference on Data Mining (ICDM)*. 859–864. https://doi.org/10.1109/ICDM.2016.0103 LCS-based streaming log parser; commonly compared with Drain..

[19] Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. 2017. DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 1285–1298. https://doi.org/10.1145/3133956.3134015 LSTM-based log-sequence anomaly detection; canonical deep-learning RCA baseline cited in failure-diagnosis surveys..

[20] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. 2023. RAGAS: Automated Evaluation of Retrieval Augmented Generation. arXiv:2309.15217 [cs.CL] https://arxiv.org/abs/2309.15217 Faithfulness, answer relevancy, context precision/recall metrics for RAG pipelines. Reference for the operationalisation of grounding metrics in our 'evidenceRefs' design..

[21] Falco Authors and Cloud Native Computing Foundation. 2026. Falco — Cloud Native Runtime Security. CNCF graduated project. https://falco.org/ eBPF-based syscall monitoring agent originally by Sysdig. Cited as orthogonal Kubernetes-runtime evidence source..

[22] Nicole Forsgren, Jez Humble, and Gene Kim. 2018. *Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations*. IT Revolution Press. Source of the four DORA metrics (deployment frequency, lead time, change-failure rate, MTTR). Cited as the standard quantitative DevOps performance framework..

[23] Milton Friedman. 1937. The Use of Ranks to Avoid the Assumption of Normality Implicit in the Analysis of Variance. *J. Amer. Statist. Assoc.* 32, 200 (1937), 675–701. https://doi.org/10.1080/01621459.1937.10503522

[24] Pinjia He, Jieming Zhu, Zibin Zheng, and Michael R. Lyu. 2017. Drain: An Online Log Parsing Approach with Fixed Depth Tree. In *Proceedings of the IEEE International Conference on Web Services (ICWS)*. 33–40. https://doi.org/10.1109/ICWS.2017.13 De facto standard online log parser, upstream of DeepLog / LogAnomaly / log-based RCA pipelines..

[25] Benjamin Hindman, Andy Konwinski, Matei Zaharia, Ali Ghodsi, Anthony D. Joseph, Randy Katz, Scott Shenker, and Ion Stoica. 2011. Mesos: A Platform for Fine-grained Resource Sharing in the Data Center. In *Proceedings of the 8th USENIX Conference on Networked Systems Design and Implementation (NSDI)*. 295–308. https://www.usenix.org/conference/nsdi11/mesos-platform-fine-grained-resource-sharing-data-center Two-level scheduler architecture pre-dating Borg / Kubernetes; cited for the cluster-management state of the art..

[26] Sture Holm. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70. https://www.jstor.org/stable/4615733 Step-down FWER correction adopted as the primary multiple-comparison control in analysis-plan.md..

[27] Jaeger Authors and Cloud Native Computing Foundation. 2026. Jaeger — End-to-End Distributed Tracing. CNCF graduated project. https://www.jaegertracing.io/ Originally Uber, donated

to CNCF; OpenTracing/OpenTelemetry-compatible trace backend deployed alongside our operator on the AKS cluster..

[28] Jenkins X Project. 2026. Preview Environments. Jenkins X Documentation. https://jenkins-x.io/

[29] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of Hallucination in Natural Language Generation. *Comput. Surveys* 55, 12, Article 248 (2023), 38 pages. https://doi.org/10.1145/3571730

[30] K8sGPT Authors and the Cloud Native Computing Foundation. 2026. K8sGPT: A Tool for Scanning Your Kubernetes Clusters, Diagnosing and Triaging Issues in Simple English. CNCF Sandbox project. https://github.com/k8sgpt-ai/k8sgpt Off-the-shelf Kubernetes RCA tool used as external baseline B2 in analysis-plan.md Section8. Rule-based scanner with LLM-based explanation. NOT a peer-reviewed comparator; pin the exact released version in every result row..

[31] Kagent Authors. 2025. Kagent: Bringing Agentic AI to Cloud Native. https://kagent.dev/ CNCF Sandbox Project (accepted May 2025).

[32] Hashir Aslam Khan, Chandra Maddila, and Thomas D. Latoza. 2024. Can GPT-4 Replicate Empirical Software Engineering Research?. In *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)*. https://2024.esec-fse.org/details/fse-2024-research-papers/7/Can-GPT-4-Replicate-Empirical-Software-Engineering-Research-Reports that GPT-4 struggles to encode SE-common-knowledge assumptions; relevant caveat for any LLM-as-judge analytical role..

[33] Avishree Khare, Saikat Dutta, Ziyang Li, Alaia Solko-Breslin, Rajeev Alur, and Mayur Naik. 2024. Reality Check: Assessing GPT-4 in Fixing Real-World Software Vulnerabilities. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering (EASE)*. https://doi.org/10.1145/3661167.3661207 Empirical evaluation of GPT-4 on real CVE-class fixes. Cited as comparable methodology for evaluating LLM diagnostic quality on real software faults..

[34] Barbara A. Kitchenham, Shari Lawrence Pfleeger, Lesley M. Pickard, Peter W. Jones, David C. Hoaglin, Khaled El Emam, and Jarrett Rosenberg. 2002. Preliminary Guidelines for Empirical Research in Software Engineering. *IEEE Transactions on Software Engineering* 28, 8 (2002), 721–734. https://doi.org/10.1109/TSE.2002.1027796 Foundational SE empirical guidelines; cited in analysis-plan.md Section8 as the standard requiring external baselines beyond internal ablations..

[35] Kubernetes SIG API Machinery. 2026. Kubebuilder: SDK for Building Kubernetes APIs Using Custom Resource Definitions. Kubernetes special-interest group project. https://book.kubebuilder.io/ Reference implementation framework for the operator pattern; the preview-operator artifact is built on Kubebuilder + controller-runtime..

[36] Timothy Lebo, Satya Sahoo, Deborah McGuinness, Khalid Belhajjame, James Cheney, David Corsar, Daniel Garijo, Stian Soiland-Reyes, Stephan Zednik, and Jun Zhao. 2013. *PROV-O: The PROV Ontology*. W3C Recommendation. World Wide Web Consortium (W3C). https://www.w3.org/TR/prov-o/ OWL2 encoding of the PROV data model; the schema target for our provenance graph (failure-evidence-model.md)..

[37] Cheryl Lee, Tianyi Yang, Zhuangbin Chen, Yuxin Su, and Michael R. Lyu. 2023. Eadro: An End-to-End Troubleshooting Framework for Microservices on Multi-source Data. In *Proceedings of the 45th International Conference on Software Engineering (ICSE)*. 1750–1762. https://doi.org/10.1109/ICSE48619.2023.00150 Multi-source (traces + logs + KPIs) anomaly detection and root-cause localization. HR@1 baseline reference..

[38] Junyi Li, Xiaoxue Cheng, Wayne Xin Zhao, Jian-Yun Nie, and Ji-Rong Wen. 2023. HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. https://arxiv.org/abs/2305.11747 35K hallucinated-vs-grounded text pairs; benchmark used to calibrate LLM-as-judge protocols. Complements FaithJudge (tamber$_faithjudge_2$025)..

[39] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, et al. 2023. Holistic Evaluation of Language Models. *Annals of the New York Academy of Sciences* (2023). https://doi.org/10.1111/nyas.15007 Stanford CRFM HELM framework: 16 scenarios x 7 metrics across 30 models. Methodological precedent for our multi-metric, multi-LLM evaluation grid..

[40] LitmusChaos Authors and Cloud Native Computing Foundation. 2026. Litmus — Cloud-Native Chaos Engineering Platform. CNCF incubating project. https://litmuschaos.io/ Kubernetes-native chaos engineering via CRDs; relevant prior art for fault injection in cloud-native environments..

[41] Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. 2023. SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. https://aclanthology.org/2023.emnlp-main.557/ Sample-consistency hallucination detection without external knowledge. Methodological precedent for the multi-sample LLM-judge protocol in analysis-plan.md RQ4..

[42] Jiya Manchanda, Laura Boettcher, Matheus Westphalen, and Jasser Jasser. 2024. The Open Source Advantage in Large Language Models. arXiv:2412.12004 [cs.CL] https://arxiv.org/abs/2412.12004 Reproducibility argument for including an open-weights LLM (Llama 3.3) alongside the closed-source baseline. Cited in llm-selection.md..

[43] Mary L. McHugh. 2012. Interrater Reliability: The Kappa Statistic. *Biochemia Medica* 22, 3 (2012), 276–282. https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3900052/ Source of the kappa interpretation thresholds used as the annotator-agreement floor in analysis-plan.md Section5..

[44] Luc Moreau, Paolo Missier, et al. 2013. PROV-DM: The PROV Data Model. W3C Recommendation. https://www.w3.org/TR/prov-dm/

[45] Hiep Nguyen, Yongmin Tan, and Xiaohui Gu. 2011. PAL: Propagation-aware Anomaly Localization for Cloud Hosted Distributed Applications. In *Proceedings of the Managing Large-scale Systems via the Analysis of System Logs and the Application of Machine Learning Techniques workshop (SLAML)*. https://doi.org/10.1145/2038633.2038634 Application-agnostic anomaly localization via critical change point ordering across components; classical reference in distributed-system RCA literature..

[46] Okteto. 2026. Preview Environments. Okteto Documentation. https://www.okteto.com/docs/

[47] OpenTelemetry Authors / CNCF. 2026. OpenTelemetry Documentation. opentelemetry.io. https://opentelemetry.io/docs/ Logs, metrics, traces; semantic conventions..

[48] OpenTelemetry Authors / CNCF. 2026. OpenTelemetry Operator for Kubernetes. GitHub: open-telemetry/opentelemetry-operator. https://github.com/open-telemetry/opentelemetry-operator Auto-instrumentation via the Instrumentation custom resource..

[49] Operator Framework Maintainers and Cloud Native Computing Foundation. 2026. Operator Framework / Operator SDK. CNCF incubating project. https://operatorframework.io/ Red Hat / CoreOS-originated framework for operator-pattern Kubernetes extensions; cited alongside Kubebuilder in the operator-pattern lineage..

[50] Luan Pham, Hongyu Zhang, Huong Ha, Flora Salim, and Xiuzhen Zhang. 2025. RCAEval: A Benchmark for Root Cause Analysis of Microservice Systems with Telemetry Data. In *Companion Proceedings of the ACM Web Conference (WWW Companion)*. https://doi.org/10.1145/3701716.3715290 735 failure cases across three microservice systems (Online Boutique, Sock Shop, Train Ticket); benchmark suites RE1/RE2/RE3; reports Top-k accuracy, Avg@5, MRR. arXiv submitted 2024-12-22..

[51] Prometheus Authors and Cloud Native Computing Foundation. 2026. Prometheus — Monitoring System and Time-Series Database. CNCF graduated project. https://prometheus.io/ De facto Kubernetes metrics monitoring stack; cited as the metrics-evidence collection precedent for CollectionOverhead measurement..

[52] Robusta Dev. 2026. HolmesGPT: AI Agent for Kubernetes Troubleshooting. Open-source project. https://github.com/robusta-dev/holmesgpt Open-source agentic ReAct-pattern Kubernetes diagnosis tool. Cited alongside K8sGPT and Kagent in analysis-plan.md Section8 as a contemporary practitioner baseline..

[53] Per Runeson and Martin Höst. 2009. Guidelines for Conducting and Reporting Case Study Research in Software Engineering. *Empirical Software Engineering* 14, 2 (2009), 131–164. https://doi.org/10.1007/s10664-008-9102-8 Companion to Kitchenham 2002 for case-study methodology; cited in threats-to-validity.md..

[54] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and

Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*. https://arxiv.org/abs/2302.04761 Self-supervised tool-use training. Cited for the agentic-tool-use lineage that HolmesGPT/Kagent build on..

[55] Malte Schwarzkopf, Andy Konwinski, Michael Abd-El-Malek, and John Wilkes. 2013. Omega: Flexible, Scalable Schedulers for Large Compute Clusters. In *Proceedings of the 8th European Conference on Computer Systems (EuroSys)*. 351–364. https://doi.org/10.1145/2465351.2465386 Shared-state cluster scheduler; cited alongside Borg/Mesos in the orchestration lineage..

[56] Lin Shi, Chiyu Ma, Wenhua Liang, Xingjian Diao, Weicheng Ma, and Soroush Vosoughi. 2024. Judging the Judges: A Systematic Study of Position Bias in LLM-as-a-Judge. arXiv:2406.07791 [cs.CL] https://arxiv.org/abs/2406.07791 15 LLM judges, 150 000 evaluations: position bias can shift accuracy >10% on pairwise code judging. Source of the LLM-as-judge bias controls in analysis-plan.md RQ4..

[57] Benjamin H. Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal, Daniel Beaver, Saul Jaspan, and Chandan Shanbhag. 2010. *Dapper, a Large-Scale Distributed Systems Tracing Infrastructure*. Technical Report dapper-2010-1. Google. https://research.google/pubs/dapper-a-large-scale-distributed-systems-tracing-infrastructure/ Foundational paper for span-based distributed tracing; intellectual basis for OpenTracing -> OpenTelemetry..

[58] Signadot. 2026. Preview Environments / Sandboxes. Signadot Documentation. https://www.signadot.com/

[59] Dag I. K. Sjøberg, Jo E. Hannay, Ove Hansen, Vigdis By Kampenes, Amela Karahasanović, Nils-Kristian Liborg, and Anette C. Rekdal. 2005. A Survey of Controlled Experiments in Software Engineering. *IEEE Transactions on Software Engineering* 31, 9 (2005), 733–753. https://doi.org/10.1109/TSE.2005.97 Quantitative survey of 103 controlled experiments in 12 leading SE venues over 1993–2002. Reference for the baseline rate of controlled-experiment reporting in SE..

[60] Manveer Singh Tamber, Forrest Sheng Bao, Chenyu Xu, Ge Luo, Suleman Kazi, Minseok Bae, Miaoran Li, Ofer Mendelevitch, Renyi Qu, and Jimmy Lin. 2025. Benchmarking LLM Faithfulness in RAG with Evolving Leaderboards. arXiv:2505.04847 [cs.CL] https://arxiv.org/abs/2505.04847 FaithJudge LLM-as-judge framework for hallucination/faithfulness evaluation. Framing reference for HallucinationRate in analysis-plan.md RQ4..

[61] The Istio Authors. 2026. Istio — An Open Source Service Mesh. Open-source project (Google, IBM, Lyft). https://istio.io/latest/about/service-mesh/ Sidecar-based service mesh with traffic management, security, and observability for microservices. Cited as the observability substrate available to the preview-operator in AKS deployments..

[62] The Kubernetes Authors. 2026. Controllers. Kubernetes Documentation. https://kubernetes.io/docs/concepts/architecture/controller/ Reconciliation / control loop concept. URL verified reachable 2026-05-22; page title "Controllers" confirmed. Living document; year set to access year..

[63] The Kubernetes Authors. 2026. Custom Resources. Kubernetes Documentation. https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/ Concepts: Custom Resource Definitions (CRDs). URL verified reachable 2026-05-22; page title "Custom Resources" confirmed. Living document; year set to access year..

[64] The Kubernetes Authors. 2026. Finalizers. Kubernetes Documentation. https://kubernetes.io/docs/concepts/overview/working-with-objects/finalizers/ Pre-deletion cleanup hook used to sequence evidence persistence. URL verified reachable 2026-05-22; page title "Finalizers" confirmed. Living document; year set to access year..

[65] The Kubernetes Authors. 2026. Jobs. Kubernetes Documentation. https://kubernetes.io/docs/concepts/workloads/controllers/job/ Run-to-completion workload pattern; standard for migration / seed / test jobs in the preview pipeline..

[66] The Kubernetes Authors. 2026. Operator pattern. Kubernetes Documentation. https://kubernetes.io/docs/concepts/extend-kubernetes/operator/ URL verified reachable 2026-05-22; page title "Operator pattern" confirmed. Living document; year set to access year..

[67] András Vargha and Harold D. Delaney. 2000. A Critique and Improvement of the CL Common Language Effect Size Statistics of McGraw and Wong. *Journal of Educational and Behavioral Statistics* 25, 2 (2000), 101–132. https://doi.org/10.3102/10769986025002101 Source of the A12 effect-size measure and the small/medium/large thresholds (0.56/0.64/0.71) adopted in analysis-plan.md Section5..

[68] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. 2015. Large-scale Cluster Management at Google with Borg. In *Proceedings of the Tenth European Conference on Computer Systems (EuroSys)*. https://doi.org/10.1145/2741948.2741964 Borg system architecture; direct intellectual ancestor of Kubernetes. Cited for the operator/controller design lineage..

[69] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *Proceedings of the Eleventh International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2203.11171 Multi-sample reasoning aggregation; relevant to discussions of LLM-judge variance reduction in analysis-plan.md RQ4..

[70] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*. https://arxiv.org/abs/2201.11903

[71] Edwin B. Wilson. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *J. Amer. Statist. Assoc.* 22, 158 (1927), 209–212. https://doi.org/10.1080/01621459.1927.10502953 Score-interval method used for all proportion CIs in metrics.md and analysis-plan.md (better small-sample coverage than the Wald interval)..

[72] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. 2012. *Experimentation in Software Engineering* (2nd ed.). Springer. https://doi.org/10.1007/978-3-642-29044-2 Reference for the threats-to-validity taxonomy (conclusion / internal / construct / external) used in threats-to-validity.md and analysis-plan.md Section4 (Threats table)..

[73] Li Wu, Jasmin Bogatinovski, Sasho Nedelkoski, Johan Tordsson, and Odej Kao. 2021. Performance Diagnosis in Cloud Microservices Using Deep Learning. In *Service-Oriented Computing (ICSOC) Workshops*. https://doi.org/10.1007/978-3-030-76352-7_13 Fine-grained root cause localization with DirectLiNGAM causal graphs and PageRank. Cited in state-of-the-art microservice RCA..

[74] Li Wu, Johan Tordsson, Erik Elmroth, and Odej Kao. 2020. MicroRCA: Root Cause Localization of Performance Issues in Microservices. In *Proceedings of the IEEE/IFIP Network Operations and Management Symposium (NOMS)*. https://doi.org/10.1109/NOMS47738.2020.9110353 Application-agnostic root cause localization correlating performance symptoms with system resource utilization; widely cited baseline in microservice RCA..

[75] Ruyue Xin, Peng Chen, and Zhiming Zhao. 2023. CausalRCA: Causal Inference Based Precise Fine-Grained Root Cause Localization for Microservice Applications. *Journal of Systems and Software* 203 (2023). https://doi.org/10.1016/j.jss.2023.111724 Gradient-based variational autoencoder causal structure learning (DAG-GNN) with PageRank for root-cause inference. Complements RCAEval/Eadro as a causal-graph baseline in state-of-the-art.md..

[76] Junjielong Xu, Qinan Zhang, Zhiqing Zhong, Shilin He, Chaoyun Zhang, Qingwei Lin, Dan Pei, Pinjia He, Dongmei Zhang, and Qi Zhang. 2025. OpenRCA: Can Large Language Models Locate the Root Cause of Software Failures?. In *Proceedings of the Thirteenth International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=M4qNIzQYpd 335 failures, >68 GB telemetry; reports Claude 3.5 best at 11.34% success — the multi-LLM evaluation precedent cited by llm-selection.md..

[77] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. https://doi.org/10.18653/v1/D18-1259 Multi-hop QA dataset with required supporting-fact annotations; the supporting-fact pattern is the QA-side analogue of our evidence-grounding constraint..

[78] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*. https://arxiv.org/abs/2305.10601 Search-based deliberative reasoning over LLM thoughts; cited to position our deterministic rule + LLM-grounded design vs frontier-style deliberation..

[79] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=WE_vluYUL-X

[80] Andreas Zeller and Ralf Hildebrandt. 2002. Simplifying and Isolating Failure-Inducing Input. *IEEE Transactions on Software Engineering* 28, 2 (2002), 183–200. https://doi.org/10.1109/32.988498 Delta debugging algorithm; intellectual ancestor of automated bisecting failure-isolation techniques in software engineering..

[81] Quanjun Zhang, Chunrong Fang, Bowen Yu, Weisong Sun, Tongke Zhang, and Zhenyu Chen. 2026. A Systematic Literature Review on Large Language Models for Automated Program Repair. *ACM Transactions on Software Engineering and Methodology (TOSEM)* (2026). https://arxiv.org/abs/2405.01466 189 papers (2020–2025) on LLM-APR; cited as state-of-the-art positioning for the LLM-diagnostic step..

[82] Shenglin Zhang, Sibo Xia, Wenzhao Fan, Binpeng Shi, Xiao Xiong, Zhenyu Zhong, Minghua Ma, Yongqian Sun, and Dan Pei. 2025. Failure Diagnosis in Microservice Systems: A Comprehensive Survey and Analysis. *ACM Transactions on Software Engineering and Methodology* 35, 1 (2025), 1–55. https://doi.org/10.1145/3715005 Survey taxonomy of failure-diagnosis techniques by data class (logs, metrics, traces, multimodal). Central reference for state-of-the-art.md positioning of operator-captured evidence..

[83] Jieming Zhu, Shilin He, Pinjia He, Jinyang Liu, and Michael R. Lyu. 2023. Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics. In *Proceedings of the IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*. https://doi.org/10.1109/ISSRE59848.2023.00071 HDFS, BGL, Thunderbird and other benchmark log datasets used by every modern log-anomaly paper..